



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details Section No.: 8 Team No.: 1

Project Title: Alarm Clock System

Team Members

Roll Number	Student Name	Signature
2520030048	Teja Hasini Reddy	
2520030237	Hansika Reddy	
2520030166	Simran	
2520030064	Anthony Nissy	

1. Abstract

Modern Linux systems often need to notify a user or trigger an action at a specific future time while continuing to perform other work — the classic requirement addressed by an alarm clock utility. This project implements a Linux-based alarm clock in C that lets a user schedule one or more alarms by specifying an absolute time or a delay, while the main process stays free to accept further commands. At the operating-system level, the project demonstrates a request's journey from user space to kernel space: a dedicated process is created with `fork()` to manage each alarm, and `alarm()`/POSIX interval timers ask the kernel to deliver a `SIGALRM` at the requested instant, which a handler installed with `sigaction()` intercepts asynchronously. Active alarms are stored in a dynamically allocated linked list (`malloc()/free()`) so multiple simultaneous requests can be tracked, matched, and cancelled on demand. A POSIX thread runs alongside the main thread to monitor pending alarms, with a mutex and condition variable protecting the shared alarm list from race conditions, and every alarm event is logged to a file using low-level Linux file I/O system calls. The expected outcome is a robust, concurrent alarm-scheduling utility that suspends and wakes processes at the correct time using signal-driven notification instead of busy-waiting, cleanly reaps every child process, and demonstrates core process, signal, memory, file, and concurrency concepts taught in the course.

2. Problem Statement

A Linux system frequently needs to perform a time-triggered action — such as reminding a user or timing a task — without blocking the rest of the system while it waits. Repeatedly checking the current time in a loop (busy-waiting) wastes CPU cycles and does not scale once several alarms must be tracked at once. This project addresses the systems-programming problem of suspending a process efficiently until a precise future instant, receiving an asynchronous kernel notification the moment that instant arrives, correctly managing multiple independent alarm requests without conflict, and allowing an active alarm to be cancelled safely — all using standard Linux/POSIX process, signal, memory, and concurrency primitives rather than GUI or polling-based application logic.

3. Objectives

1. To design and implement a Linux-based alarm clock that accepts an absolute time or a relative delay from the user and schedules the alarm using POSIX system calls.
2. To create a dedicated process for each alarm using `fork()`, and to use `alarm()`/POSIX timers together with a `SIGALRM` handler installed via `sigaction()` to notify the process asynchronously when the alarm time is reached.
3. To manage multiple simultaneous alarm requests using a dynamically allocated linked-list data structure and to allow the user to cancel any active alarm before it fires.
4. To synchronize alarm-monitoring activity using POSIX threads, mutexes and condition variables, and to maintain a persistent log of alarm events using Linux file I/O system calls.

4. Proposed Methodology

The system is implemented in C on Ubuntu Linux using only user-space Linux/POSIX APIs. The main process presents a menu-driven interface that accepts an alarm time or a delay in seconds, computes the wait interval, and inserts the request into a dynamically allocated node in a linked list of active alarms. For every accepted request, the parent uses `fork()` to create a child process that calls `alarm()` (or `timer_create()/timer_settime()` when several alarms run concurrently) so the kernel delivers a `SIGALRM` at the requested instant, then suspends itself using `pause()` while the parent continues to accept new requests or cancellations. A handler registered with `sigaction()` intercepts `SIGALRM` asynchronously, prints an alarm notification, updates the alarm's status in the shared list, and the child exits cleanly; the parent reaps it with `waitpid()` to avoid zombie processes. A background POSIX thread, synchronized with the main thread through a mutex and a condition variable, periodically checks the alarm list for expired or cancelled entries and keeps the display consistent. All alarm events — creation, firing, cancellation, and errors — are written to a log file using `open()`, `write()`, and `close()`. The design is validated using `strace` to confirm the sequence of system calls and `gdb/Valgrind` to verify correct process behavior and leak-free memory handling.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
System calls (<code>alarm()</code> , <code>sleep()</code> , <code>open()</code> , <code>write()</code>)	Demonstrate the transition of an alarm request from user space to kernel-space timer and I/O services.
User space vs. kernel space, mode switching	Illustrate how the alarm-setting request and the resulting <code>SIGALRM</code> delivery cross the user/kernel boundary.
Process creation using <code>fork()</code>	Create a dedicated child process to independently manage each scheduled alarm.
Process termination and reaping using <code>wait()/waitpid()</code>	Collect the exit status of alarm-handling child processes and prevent zombie processes.
Signals and asynchronous notification (<code>SIGALRM</code>) via <code>alarm()</code> /POSIX timers	Generate an asynchronous kernel notification exactly when the requested alarm time is reached.
Signal handling using <code>sigaction()</code>	Reliably intercept <code>SIGALRM</code> and trigger the alarm notification and cleanup logic.
Dynamic memory allocation (<code>malloc()/realloc()/free()</code>)	Maintain a linked list of active alarm requests that can grow and shrink at runtime.
Memory debugging using Valgrind	Verify that alarm nodes are correctly freed and no memory leaks occur across repeated create/cancel cycles.
Linux file I/O system calls (<code>open()</code> , <code>write()</code> , <code>close()</code>)	Persist a log of alarm creation, firing, and cancellation events to a file.
File descriptors	Manage the log file and standard I/O streams used by the alarm process and its parent.

OS Concept / Linux API / System Call	Purpose in the Project
POSIX threads (pthread_create()/pthread_join())	Run a background thread that monitors pending alarms concurrently with the main input loop.
Mutexes and condition variables	Synchronize access to the shared alarm list between the main thread, monitor thread, and signal handler to avoid race conditions.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030048	Teja Hasini Reddy	Designed the input-handling module and alarm-scheduling logic; implemented fork()-based child process creation for each alarm request.
2520030237	Hansika Reddy	Implemented signal generation and handling (alarm(), sigaction(), SIGALRM handler) and the alarm-cancellation feature.
2520030166	Simran	Implemented dynamic memory management for the active-alarm linked list and carried out Valgrind-based memory testing.
2520030064	Anthony Nissy	Implemented POSIX-thread-based alarm monitoring with mutex/condition-variable synchronization, file-based event logging, and test documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution platform providing all Linux/POSIX system calls used in the project.
C / C++	Implementation language for process, signal, memory, file, and thread management logic.
GCC	Compiler used to build the alarm clock system, driven through a project Makefile.
GDB and strace	Debugging and system-call tracing tools used to verify process, signal, and I/O behavior.
Valgrind	Detects memory leaks and memory errors in the dynamically allocated alarm data structures.
Makefile	Automates compilation and linking of all project source files.

8. Expected Outcome

The completed system is expected to reliably schedule and fire single or multiple alarms at the correct time using signal-based notification instead of CPU-wasting busy-waiting, to let the user view and cancel pending alarms at any point, and to terminate all child processes cleanly without leaving zombies. It will produce a readable event log confirming correct alarm creation, firing, and cancellation, along with strace traces and a Valgrind report demonstrating correct system-call usage and leak-free dynamic memory management. Final deliverables include the working C source code, Makefile, README, test cases, and output evidence as required by the project guidelines.

9. GitHub Repository

GitHub Repository Link: : https://github.com/7Hansika7/Tiramisu_OSSP

Repository Created : [7Hansika7/Tiramisu OSSP](https://github.com/7Hansika7/Tiramisu_OSSP)

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr. Muniraju Naidu

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____